Практическая работа 12

Логическое и физическое копирование (резервирование) в PostgreSQL

Логическое резервирование

Логическое резервирование позволяет сохранять SQL-команды, выполнив которые можно восстановить базу данных, восстановив объекты. У логического копирования есть недостаток – это невысокая скорость. В то же врем полученную резервную копию можно использовать для долговременного хранения.

Команда copy.

Если требуется сохранить только содержимое одной таблицы, можно воспользоваться командой COPY.

Команда позволяет записать таблицу (или часть столбцов таблицы, или даже результат произвольного запроса) либо в файл, либо на консоль, либо на вход какой-либо программе. При этом можно указать ряд параметров, таких как формат (текстовый, CSV или двоичный), разделитель полей, текстовое представление NULL и т. п.

Другой вариант команды, наоборот, считывает из файла или из консоли строки с полями и записывает их в таблицу. Таблица при этом не очищается, новые строки добавляются к уже существующим. Команда COPY работает существенно быстрее, чем аналогичные команды INSERT — клиенту не нужно много раз обращаться к серверу, а серверу не нужно много раз анализировать команды.

Тонкость: при выполнении команды COPY FROM не применяются правила (rules), хотя ограничения целостности и триггеры выполняются.

В psql существует клиентский вариант команды COPY с аналогичным синтаксисом. В отличие от серверного варианта COPY, который является командой SQL, клиентский вариант — это команда psql. Указание имени файла в команде SQL соответствует файлу на сервере БД. У пользователя, под которым работает PostgreSQL (обычно postgres), должен быть доступ к этому файлу. В клиентском же варианте обращение к файлу происходит на клиенте, а на сервер передается только содержимое.

Утилита pg_dump

Для создания полноценной резервной копии базы данных используется утилита pg_dump. Если не указать имя файла (-f, --file), то утилита выведет результат на консоль. А результатом является скрипт, предназначенный для psql, который содержит команды для создания необходимых объектов и наполнения их данными.

Дополнительными ключами утилиты можно ограничить набор объектов: выбрать указанные таблицы, или все объекты в указанных схемах, или наложить другие фильтры. Чтобы восстановить объекты из резервной копии, достаточно выполнить полученный

скрипт в psql. Следует иметь в виду, что базу данных для восстановления надо создавать из шаблона template0, так как все изменения, сделанные в template1, также попадут в резервную копию. Кроме того, заранее должны быть созданы необходимые роли и табличные пространства. Поскольку эти объекты не относятся к конкретной БД, они не будет выгружены в резервную копию. После восстановления базы имеет смысл выполнить команду ANALYZE: она соберет статистику, необходимую оптимизатору для планирования запросов.

Утилита pg_dumpall

Чтобы создать резервную копию всего кластера, включая роли и табличные пространства, можно воспользоваться утилитой pg_dumpall. Поскольку pg_dumpall требуется доступ ко всем объектам всех БД, имеет смысл запускать ее от имени суперпользователя. Утилита по очереди подключается к каждой БД кластера и выгружает информацию с помощью pg_dump. Кроме того, она сохраняет и данные, относящиеся к кластеру в целом. Чтобы начать работу, утилите требуется подключиться хотя бы к какойто базе данных. По умолчанию выбирается postgres или template1, но можно указать и другую. Результатом работы pg_dumpall является скрипт для psql. Другие форматы не поддерживаются. Это означает, что pg_dumpall не поддерживает параллельную выгрузку данных, что может оказаться проблемой при больших объемах данных. В таком случае можно воспользоваться ключом --globals-only, чтобы выгрузить только роли и табличные пространства, а сами базы данных выгрузить отдельно с помощью pg_dump в параллельном режиме.

Нелогично использовать логическое резервирование при малых потерях данных.

Пример.

Взять любую базу, из имеющихсч на сервере. Убедиться, что база заполнена информацией:

```
for_back=# select * from test;
 id |              s
----+-----------------------------
  1 | Мария
  2 | Тест
  3 | Добрый вечер!
```

Рисунок 1 – Выполнение тестового запроса

Далее выполняется копирование в выходной поток с помощью STDOUT

```
for_back=# copy test to stdout;
1       ? аЁп
2       '?бв
3       "RЎалс ў?з?а!
```

Рисунок 2 – Вывод значений из таблицы в поток

Далее следует очистка и возможность копирования из потока в таблицу.

```
for_back=# copy test from stdin;
Вводите данные для копирования, разделяя строки переводом строки.
Закончите ввод строкой '\.' или сигналом EOF.
>> 1    Мария
>> 2    Тест
>> 3    Добрый вечер!
>> \.
COPY 3
for_back=# select * from test;
 id |               s
----+-------------------------------
  1 | Мария
  2 | Тест
  3 | Добрый вечер!
```

Рисунок 3 – Добавление записей

При успешной вставке будет отображен результат. В конце копирования обязательно указать символ \.

Также с помощью другой команды \copy можно скопировать данные в файл.

```
for_back=# \copy test to '/users/damse/documents/test.csv';
COPY 3
for_back=#
```

Рисунок 4 – Копирование данных из таблицы в файл csv

В случае удачного копирования появится сообщение о том какие строки были скопированы (в каком количестве).

Задание.

Скопировать из тестовой базы информацию в выходной потом, затем очистить из добавить из входного потока. Записать, информацию, содержащуюся в таблице в новый файл (csv или txt). Записать базу в gzip-архив (подсказка: воспользоваться оператором COPY совместно с PROGRAM) – задание только для тех, кто работает в UNIX-системах.

Утилиты pg_dump и pg_basebackup

Копирование также возможно с помощью утилиты pg_dump. Для корректной работы стоит обратиться к командной строке (терминалу).

```
C:\Program Files\PostgreSQL\15\bin>pg_dump -U postgres -W -d for_back --create
Пароль:
```

Рисунок 5 – Создание копии базы данных

Необходимо зайти в папку bin PostgreSQL, чтобы запустить утилиту, отвечающую за резервное копирование – pg_dump. Далее после -U прописан пользователь, -W предполагает запрос пароля (указан ниже), -d позволяет обратиться к указанной базе данных, опция — create добавляет команды для последующего создания базы данных при загрузки из дампа. Без этой опции вначале пришлось бы создать базу а потом в неё загружать дамп.

```
--
-- Name: for_copy; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.for_copy (
    id numeric,
    name character(50)
);


ALTER TABLE public.for_copy OWNER TO postgres;

--
-- Name: test; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.test (
    id numeric,
    s character(30)
);


ALTER TABLE public.test OWNER TO postgres;

--
-- Data for Name: for_copy; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY public.for_copy (id, name) FROM stdin;
\.


--
-- Data for Name: test; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY public.test (id, s) FROM stdin;
1       P?B P°P?Pї
2       в?TT?P+P?
3       в??BRP?P°P>Bc C?T?P·T?P°!
\.
```

Рисунок 6 – Логическая копия базы в командной строке

Таким образом, в терминале представлена логическая копия базы for_back.

С помощью pg_dump можно перенести данные в другие базы данных или в файл.

```
C:\Program Files\PostgreSQL\15\bin>pg_dump -U postgres -W -d for_back --table=test | psql -d backup
Пароль: Пароль пользователя damse:

admin

SET
SET
SET
SET
SET
 set_config
------------

(1 строка)

SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
COPY 3

C:\Program Files\PostgreSQL\15\bin>
```

Рисунок 7 – Запись из одной таблицы базы в другую таблицу другой базы данных

Выполнив указанные команды в базе backup создается таблица test, в которой будет храниться информация из таблицы test for_back. Для проверки можно вернуться в Shell (psql) или pgadmin4, чтобы отобразить скопированные записи.

```
backup=# select * from test
backup-# ;
 id |                s
----+--------------------------------
  1 | Мария
  2 | Тест
  3 | Добрый вечер!
```

Рисунок 8 – Проверка записи

Теперь поработаем с pg_basebackup и протоколом репликации. Убедимся что wal_level=replica и max_wal_senders больше нуля:

```
backup=# select name, setting from pg_settings where name in ('wal_level', 'max_wal_senders');
      name       | setting
-----------------+---------
 max_wal_senders | 10
 wal_level       | replica
```

Рисунок 9 – Возможность создания реплики

Убедимся что в pg_hba.conf есть разрешение подключаться по протоколу репликации:

```
postgres=# select type, database, user_name, address, auth_method from pg_hba_FILE_RULES where database='{replication}';

 type  |   database    | user_name |  address  |  auth_method
-------+---------------+-----------+-----------+---------------
 local | {replication} | {all}     |           | scram-sha-256
 host  | {replication} | {all}     | 127.0.0.1 | scram-sha-256
 host  | {replication} | {all}     | ::1       | scram-sha-256
```

Рисунок 10 – Проверка возможности подключаться по протоколу

Подключаться можно, теперь необходимо знать адрес локального хоста. Для этого в папке data найти файл pg_hba.conf (или на скриншоте выше) и просмотреть там адрес хоста для последующего копирования.

```
C:\Program Files\PostgreSQL\15\bin>pg_basebackup --host 127.0.0.1 --pgdata=/
users/damse/documents/data/
```

Рисунок 11 – Копирование содержимого хоста в пустой каталог

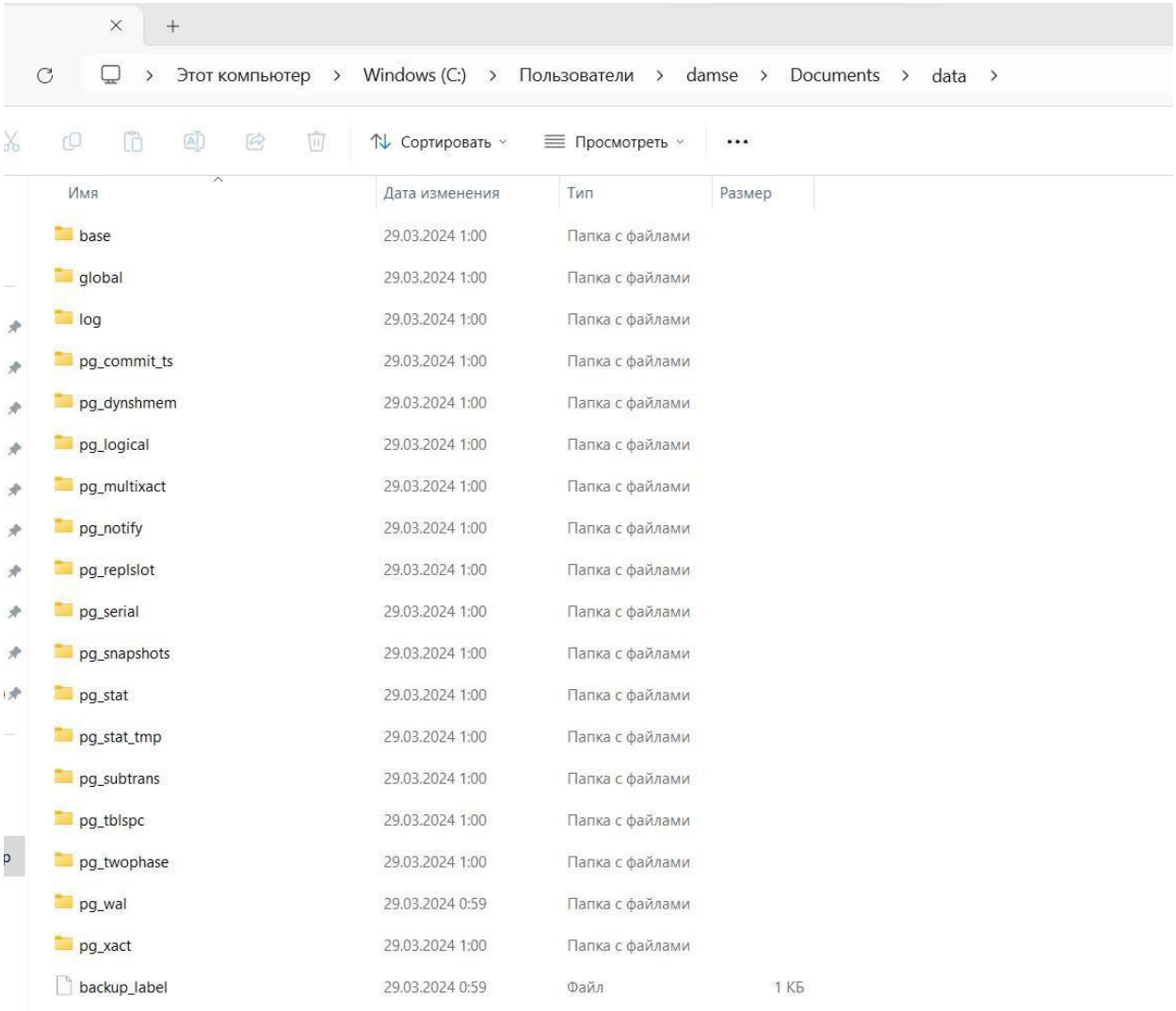


Рисунок 12 – Перемещенные файлы

Задание.

Скопировать данные в логическую копию в командную строку (терминал) и в отдельную таблицу. Попробовать выгрузить дамп из PostgreSQL и загрузить его в MySQL (phpmyadmin). Скопировать данные хоста в пустой каталог.